



OBJECT ORIENTED SYSTEMS (OOS) — 2025

QUESTION 4 COMPLETE DETAILED SOLUTION



Introduction

Question 4 of the 2025 Object Oriented Systems examination belongs to CO4 and focuses entirely on the concepts of:

- 🔥 UML (Unified Modeling Language)
- 🔥 Class Diagrams
- 🔥 Sequence Diagrams
- 🔥 Aggregation
- 🔥 Composition
- 🔥 Generalization
- 🔥 Inheritance
- 🔥 Include and Extend Relationships
- 🔥 Multiplicity
- 🔥 Roles in UML

These concepts are extremely important because UML acts as the blueprint of software systems 🚀. Before building large software applications such as banking systems, hospital management systems, e-commerce platforms, enterprise resource planning software, and distributed systems, software engineers first create UML models to:

- visualize structure,
- understand relationships,
- design interactions,
- and communicate architecture clearly 🗣️.

UML improves software planning, maintainability, scalability, debugging, and team communication. Thus this question evaluates whether the student understands software modeling principles, object relationships, and visual system representation 🧠.



QUESTION 4(a)



Illustrate how the different parts of a class are represented in a class diagram.



Answer

A **Class Diagram** is one of the most important structural diagrams in UML 🚀. It represents classes, attributes, methods, relationships, and object-oriented architecture. Class diagrams are used to visualize the static structure of a software system.



Basic Structure of UML Class Diagram

A UML class diagram mainly contains three sections:

Section	Purpose
Class Name	Identifies the class
Attributes	Represents data members
Methods	Represents member functions

🌟 UML Representation of a Class

```
-----  
|      Student      |  
-----  
| - roll : int      |  
| - name : String   |  
-----  
| + display() : void |  
| + getData() : void |  
-----
```

🌟 Deep Explanation of Each Part

📄 1 Class Name Section

The top portion contains the class name (e.g., **Student**). Class names are usually written centered, bold, and starting with a capital letter.

📄 2 Attribute Section

The middle portion contains data members, instance variables, and class attributes. *Example:*

```
- roll : int  
- name : String
```

Visibility Symbols:

Symbol	Meaning
+	Public
-	Private
#	Protected

Thus, `- roll : int` means a **private** integer variable.

📄 3 Method Section

The bottom section contains member functions. *Example:*

```
+ display() : void
```

This means a **public** method with no return value (**void**).

🌟 Complete Example with Explanation

Consider the following Java class:

```
class Student {
    private int roll;
    private String name;

    public void display() { }
    public void getData() { }
}
```

Its UML representation becomes:

```

-----
|           Student           |
|-----|
| - roll : int                |
| - name : String             |
|-----|
| + display() : void          |
| + getData() : void          |
|-----|

```

🌟 Why Class Diagrams are Important

Class diagrams help software developers visualize architecture, understand relationships, simplify maintenance, and improve communication. Large enterprise systems may contain hundreds of interacting classes. Without UML diagrams, system complexity becomes difficult to manage ⚠️.

🎯 Conclusion

A UML class diagram represents the class name, attributes, methods, and visibility using structured rectangular notation and standardized UML symbols 🚀.

📄 QUESTION 4(b)(i)

? **Distinguish between Aggregation and Composition.**

✅ Answer

Aggregation and Composition are both special forms of association relationships in UML 🚀. Both represent a "HAS-A" relationship, but they differ significantly in ownership, lifecycle dependency, and object existence.

☀️ Aggregation

Aggregation represents **weak ownership**. In aggregation, child objects can exist independently even if the parent object gets destroyed. It is represented using a hollow diamond ◇.

- **Example of Aggregation:** A **Department** has **Teachers**. Even if the department closes, teachers may still exist.
- **UML Representation:** **Department** ◇ — **Teacher**

☀️ Composition

Composition represents **strong ownership**. In composition, the child object cannot exist independently. If the parent gets destroyed, the child also gets destroyed ⚠️. It is represented using a filled diamond ◆.

- **Example of Composition:** A **House** contains **Rooms**. If the house is destroyed, the rooms also disappear.
- **UML Representation:** **House** ◆ — **Room**

☀️ Comparison Table

Feature	Aggregation	Composition
Ownership	Weak	Strong
Dependency	Independent	Dependent
Lifecycle	Separate	Shared
UML Symbol	Hollow Diamond (◇)	Filled Diamond (◆)

🎯 Conclusion

Aggregation represents weak ownership whereas composition represents strong lifecycle-dependent ownership 🚀.

📄 QUESTION 4(b)(ii)

? **Distinguish between Synchronous and Asynchronous transmission in Sequence Diagram.**

✅ Answer

In UML Sequence Diagrams, messages between objects may occur in two ways: synchronous transmission and asynchronous transmission 🚀. These represent how objects communicate during execution.

☀️ Synchronous Transmission

In synchronous communication, the sender waits until the receiver completes the operation. Thus execution becomes sequential.

- **Example:** **Client**  **Server** (waits for response)
- **Real-world Example:** Phone call 📞 (caller waits for response).

☀️ Asynchronous Transmission

In asynchronous communication, the sender does not wait. Execution continues independently.

- **Example:** **Client**  **Server** (no waiting)
- **Real-world Example:** Email 📧 (sender continues work without waiting).

☀️ Comparison Table

Feature	Synchronous	Asynchronous
Waiting	Yes	No
Blocking	Blocking	Non-blocking
Execution	Sequential	Parallel
Speed	Slower	Faster

🎯 Conclusion

Synchronous transmission blocks the sender until a response arrives, whereas asynchronous transmission allows parallel independent execution 🚀.

📄 QUESTION 4(b)(iii)

? Distinguish between Include and Extend relations.

✅ Answer

Include and Extend are relationships used in UML Use Case Diagrams 🚀. They describe interaction reuse and behavior extension.

☀️ Include Relationship

The include relationship represents **mandatory reuse**. One use case always includes another use case. Represented using `<<include>>`.

- **Example:** In an ATM system, *Withdraw Money* includes *Authenticate User*. Authentication always occurs.

☀️ Extend Relationship

The extend relationship represents **optional behavior extension**. Additional behavior executes only under certain conditions. Represented using `<<extend>>`.

- **Example:** In Online Shopping, *Apply Discount* extends *Checkout*. Discount applies conditionally.

☀️ Comparison Table

Feature	Include	Extend
Nature	Mandatory	Optional
Reuse	Common functionality	Additional behavior
Execution	Always	Conditional

Conclusion

Include relationships represent mandatory reusable behavior whereas extend relationships represent optional conditional behavior .

QUESTION 4(b)(iv)

Distinguish between Generalization and Inheritance.

Answer

Generalization and inheritance are closely related concepts in Object-Oriented Modeling . Inheritance belongs to programming implementation, while Generalization belongs to UML conceptual modeling.

Inheritance

Inheritance is a programming mechanism where a child class acquires parent properties.

- **Example:**

```
class Animal {}
class Dog extends Animal {}
```

Generalization

Generalization represents the parent-child abstraction relationship in UML. It models the “is-a” relationship.

- **UML Representation:**

```
Animal
  ▲
  |
Dog
```

Comparison Table

Feature	Generalization	Inheritance
Type	UML Concept	Programming Feature

Feature	Generalization	Inheritance
Purpose	Modeling	Code Reuse
Level	Design Level	Implementation Level

Conclusion

Generalization is UML-level abstraction whereas inheritance is programming-level implementation 🚀.

QUESTION 4(c)

? **Discuss multiplicity and role related to class diagram of UML.**

✓ Answer

Multiplicity and role are important concepts in UML Class Diagrams because they describe object participation, relationship constraints, and interaction semantics 🚀. These concepts help software designers understand how many objects participate and what responsibilities they perform.

☀ Multiplicity

Multiplicity specifies the number of objects participating in a relationship. It defines the minimum and maximum object count.

☀ Common Multiplicity Values:

Multiplicity	Meaning
1	Exactly one
0..1	Zero or one
*	Many
1..*	One or many

- **Example:**

Teacher 1 — * Student

Meaning: One teacher teaches many students.

☀ Role in UML

Role describes the responsibility of an object within a relationship.

- **Example:**

Teacher — Student
(teaches)

Here, the teacher's role = teaches.

🌟 Deep Conceptual Analysis

Multiplicity improves database modeling, object design, and system validation. Roles improve semantic clarity, interaction understanding, and relationship interpretation 🚀.

🌟 Real-world Example

In a Banking System:

Customer 1 — * Account

Meaning: One customer may own multiple accounts.

🎯 Conclusion

Multiplicity defines participation count in relationships whereas role defines the responsibility played by objects inside UML class diagrams 🚀.